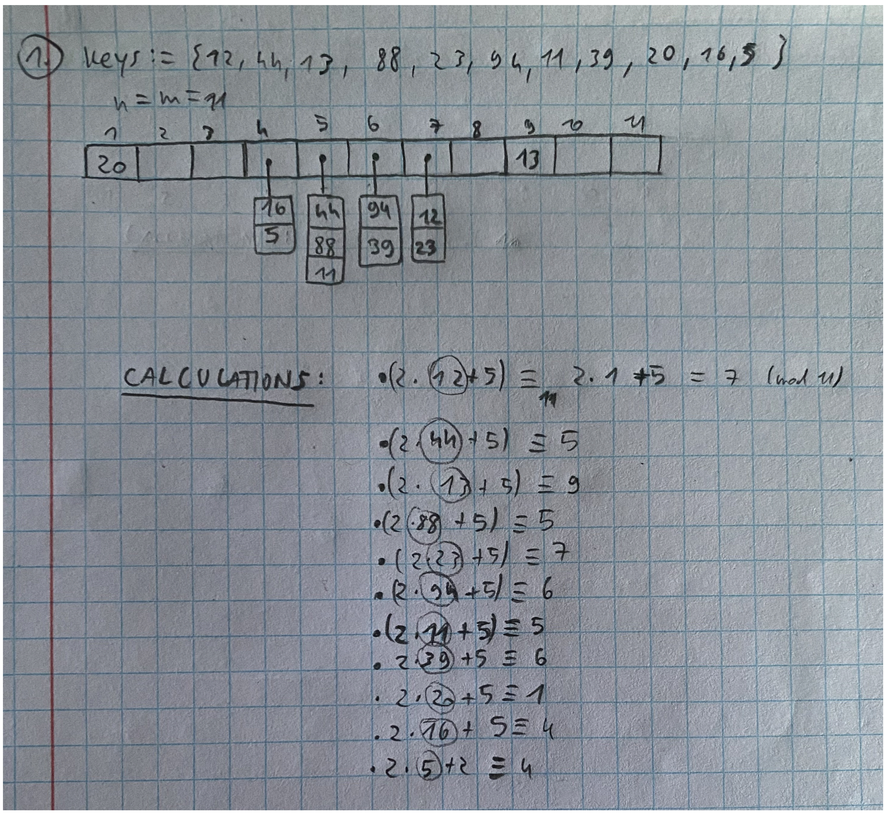
Assignment 13

Author: Lan Stare

Date: 9. 12. 2025

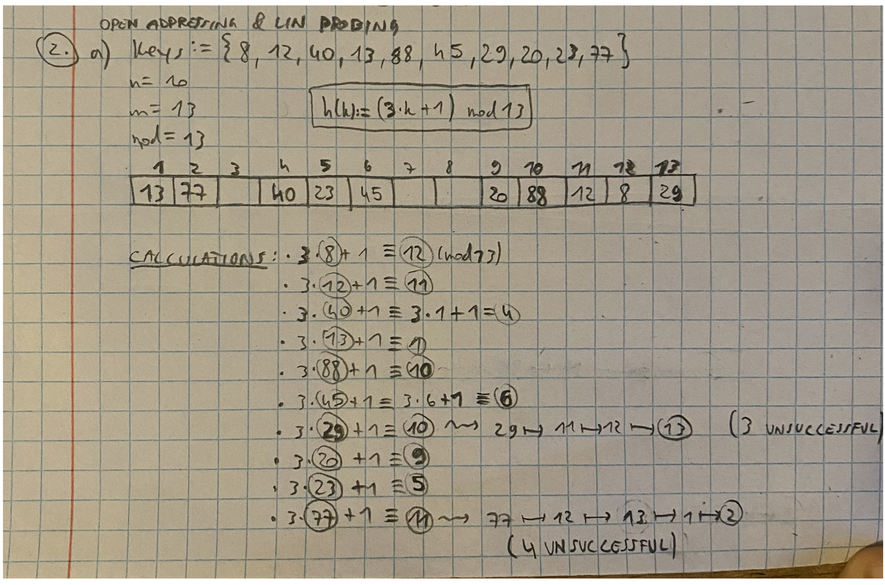
Question 1



Question 2

(a):

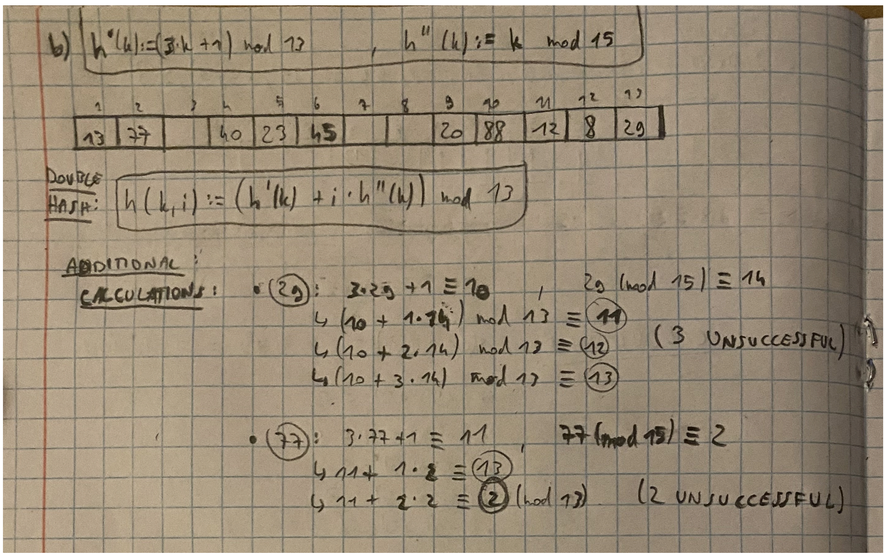
There were 7 unsuccessful probings in total:



(b)

I understood double hashing as taking the function: $h(k,i) = (h'(k) + i * h''(k)) \mod m$ starting from $i = 0$. So all the elements before 29 only get hashed by $h'(k)$. Please tell me whether the first or the second understanding is correct.

The total number of unsuccessful probings was 5, which is better than in (a):



Remark: I know I should have taken elements from $0, 1, \ldots, (m-1)$ but I realized my mistake too late. I hope I still managed not to make a significant mistake because of it.

Question 3

In most cases the first hash function: h_1 would be better, since in h_2 we would only fill 3 slots of the table, which are exactly 0, 1 and 2. The reason for this is that all elements in $1, 2, \ldots, 9$ would map to 0. All the elements in $10, 11, \ldots, 19$ would map to 1 and all the elements in $20, 21, \ldots, 29$ would map to 2; which exhausts all possibilities. However in h_1 we would get an even distribution over the table.

Question 4

EXAMPLE:

Let's take the example from a) and delete the element that is currently in slot 12 (so the number 8). If we do that, slot 12 becomes empty. If we now try searching for the number 29, which is in the slot 13, we wouldn't find it. The reason for this is that number 29 should be in slot 10 but because slots 10, 11 and 12 were all already taken, it was assigned to slot 13. We would begin searching for 29 in the slot 10 and then move down until we found 29 or an empty slot which would mean that the element does not exist in the table. Because slot 12 is empty, the search would stop there and return a wrong answer.

Question 5

Idea: We insert each element of A into a hash table T of size n using chaining (or open addressing). If an insertion finds an already taken slot, A contains a duplicate; otherwise continue. Because we can assume that we have simple uniform hashing the expected cost for insertion or lookup is $O(1)$, so total expected time is $O(n)$.

Correctness: If any element appears twice, we'll find that place already taken when we try to insert it. If no two elements are the same, the insertion succeeds without probes.